P2596R0 critique

28-04-2023

Audience: LEWG, SG14, WG21

Document number: P2857R0

Reply-to: Matthew Bentley <mattreecebentley@gmail.com>

I have been asked to provide a textual form of the issues with Arthur O'Dwyer's paper. This is it. The critique follows Arthur's paper, crosses out the sections which are false, and annotates with my comments in red and in blue.

Abstract

P0447R20 `std::hive` proposes a complicated capacity model involving the library identifiers `hive_limits`, `block_capacity_limits`, `block_capacity_hard_limits`, `reshape`, in addition to `capacity`, `reserve`, `shrink_to_fit`, and `trim_capacity`. P0447R20's model permits the user to specify a max block size~~; this causes "secret quadratic behavior" on some common operations.~~ P0447R20's model requires the container to track its min and max block sizes as part of its ~~(non-salient)~~ state.

We propose a simpler model that involves the library identifiers `max_block_size` and `reshape`, in addition to `capacity`, `reserve`, `shrink_to_fit`, and `trim_capacity`. This model does not permit the user to specify a max block size~~, so "secret quadratic behavior" is less common~~. This model does not require the container to track anything; the new `reshape` is a simple mutating operation analogous to `reserve` or `sort`.\

Matt: The user-defined block capacities are relevant to both performance and memory usage in hive. I would include them along with the O(1) skipping mechanism and erasure location recording mechanism as salient features, in that respect. This is explained in the paper, which Arthur has otherwise taken great pains to read. The various ways in which they are beneficial are explained in the summary at the bottom of this document.

There is no secret quadratic behaviour in the container implementation and never has been. The so-called behaviour was (and no longer is) O(n) in the number of blocks, when an erasure causes

a block to become empty of elements, but this behaviour no longer exists in the implementation, which was true at the time I received this paper. Further, Arthur's suggested alternative would not have removed this behaviour, as it relates primarily to block numbering, not how many blocks exist. The only it did was prevent the user from increasing the number of blocks.

§ 2. Introduction: P0447R20's model

P0447R20 `hive` is a bidirectional container~~, basically a "rope with holes."~~ Compare it to the existing STL sequen-ce containers:

Matt: Arthur has done a lot of work testing the implementation and knows the structure to be linked list of blocks (although this is not the only way of implementing it). A rope is a binary tree of substrings. A hive cannot be implemented in this way. They are entirely dissimilar containers. This appears to be an attempt to create FUD.

- `vector` is a single block with spare capacity only on the end. Only one block ever exists at a time.
- `list` is a linked list of equal-sized blocks, each with capacity 1; unused blocks are immediately deallocated.
- `deque` is a vector of equal-sized blocks, each with capacity N; spare capacity exists at the end of the last block *and* at the beginning of the first block, but nowhere else.
- `hive` is a linked list of variably-sized blocks; spare capacity ("holes") can appear anywhere in any block, and the container keeps track of where the "holes" are.

The blocks of a `hive` are variably sized. The intent is that as you insert into a hive, it will allocate new blocks of progressively larger sizes — just like `vector`'s geometric resizing, but without relocating any existing elements. This improves cache locality when iterating.

We can represent a vector, list, or hive diagrammatically, using `[square brackets]` to bracket the elements belonging to a single block, and `_` to represent spare capacity ("holes"). There's a lot of bookkeeping detail not captured by this representation; that's okay for our purposes.

```
std::vector<int> v = {1,2,3,4,5,6}; // [1 2 3 4 5 6 _ _]
std::list<int>   l = {1,2,3,4,5,6}; // [1] [2] [3] [4] [5] [6]
std::hive<int>   h = {1,2,3,4,5,6}; // [1 2 3 4 5 6]
```

Erasing from a vector shifts the later elements down. Erasing from a list or hive never needs to shift elements.

```
v.erase(std::next(v.begin()));  // [1 3 4 5 6 _ _ _]
l.erase(std::next(l.begin()));  // [1] [3] [4] [5] [6]
h.erase(std::next(h.begin()));  // [1 _ 3 4 5 6]
```

Reserving in a vector may invalidate iterators, because there's no way to strictly "add" capacity. Reserving in a hive never invalidates iterators (except for `end()`), because we can just add new blocks right onto the back of the container. (In practice, `hive` tracks unused blocks in a separate list so that iteration doesn't have to traverse them; this diagrammatic representation doesn't capture that detail.)

```
v.reserve(10);  // [1 3 4 5 6 _ _ _ _ _]
h.reserve(10);  // [1 _ 3 4 5 6] [_ _ _ _]
```

P0447R20 allows the programmer to specify `min` and `max` block capacities, via the `std::hive_limits` mechanism. No block in the hive is ever permitted to be smaller than `min` elements in capacity, nor greater than `max` elements in capacity. For example, we can construct a hive in which no block's capacity will ever be smaller than 4 or greater than 5.

```
std::hive<int> h({1,2,3,4,5,6}, std::hive_limits(4, 5));
               // [1 2 3 4 5] [6 _ _ _]
h.reserve(10); // [1 2 3 4 5] [6 _ _ _] [_ _ _ _]
```

A hive can also be "reshaped" on the fly, after construction. In the following example, after creating a hive with one size-5 block, we `reshape` the hive to include only blocks of sizes between 3 and 4 inclusive. This means that the size-5 block is now "illegal"; its elements are redistributed to other blocks, and then it is deallocated, because this hive is no longer allowed to hold blocks of that size.

```
std::hive<int> h({1,2,3,4,5,6}, std::hive_limits(4, 5));
               // [1 2 3 4 5] [6 _ _ _]
h.reserve(10); // [1 2 3 4 5] [6 _ _ _] [_ _ _ _]
h.reshape(std::hive_limits(3, 4));
               // [6 1 2 3] [4 5 _ _]
```

Notice that `reshape` invalidates iterators (to element `1`, for example), and can also undo the effects of `reserve` by reducing the overall capacity of the hive. (Before the reshape operation,

`h.capacity()` was 13; afterward it is only 8.) Programmers are advised to "`reshape` first, `reserve` second."

§ 3. Criticisms of P0447R20's model

~~3.1. Max block size is not useful in practice~~

One of the primary motivations for `hive` is to be usable in embedded/low-latency situations, where the programmer might want fine control over the memory allocation scheme. So at first glance it makes sense that the programmer should be able to specify min and max block capacities via `hive_limits`. However:

- ~~A programmer is likely to care about *min* block capacity (for cache locality), but not so much *max* capacity. The more contiguity the better! Why would I want to put a cap on it?~~

  Matt: Arthur has been given several reasons why max capacity is important in the reflector and on the github issues section for the reference implementation. In addition many have been included in the FAQ of the present paper, and in the publicly-available drafts of that paper, and some were included in the previous version of this paper (19), which evidently he has read as is reflected in the reflector discussions. These arguments and others are summarised in blue at the bottom of this paper. He has not provided any rebuttal for those reasons on the reflector or github, and none are referenced or rebutted here.
  https://github.com/mattreecebentley/plf_hive/issues/22
  https://isocpp.org/files/papers/P0447R20.html#faq, items 11 & 12
  https://isocpp.org/files/papers/P0447R19.html#faq, item 6
  See: Lewg reflector, 4/05/2022, 1:42 pm, My reply Re: [isocpp-lib-ext] Notes on P0047R20, beginning with "On 29/04/2022 3:49 am, Arthur O'Dwyer wrote:" (https://lists.isocpp.org/lib-ext/2022/05/23129.php).

- If the embedded programmer cares about max capacity, it's likely because they're using a slab allocator that hands out blocks of some fixed size (say, 1024 bytes).

  Matt: They will also be interested in memory waste based on their erasure and insertion patterns and performance. See summary.

  But that doesn't correspond to `hive_limits(0, 1024)`. The min and max values in `hive_limits` are *counts of elements*, not the size of the actual allocation. So you might try

dividing by `sizeof(T)`; but that still won't help, for two reasons:

- Just like with `std::set`, the size of the allocation is not the same as `sizeof(T)`. In the reference implementation, a block with capacity `n` typically asks the allocator for `n*sizeof(T) + n + 1` bytes of storage, to account for the skipfield structure. In my own implementation, I do a `make_shared`-like optimization that asks the allocator for `sizeof(GroupHeader) + n*sizeof(T) + n + 1` bytes.

  Matt: This is easy to work around via a static member function, and one such has been provided in the most recent versions of plf::colony for exactly this purpose ("max_elements_per_allocation", 6 lines of actual code). However I think it an edge-case for a standard library implementation, and possibly not worth including in std::hive? Open to suggestion.

- ~~The reference implementation doesn't store `T` objects contiguously, when `T` is small. When `plf::hive<char>` allocates a block of capacity `n`, it actually asks for `n*2 + n + 1` bytes instead of `n*sizeof(char) + n + 1` bytes.~~

  Matt: Arthur has been informed both on the reflector and the reference implementation's github that this situation is only for types with sizes equal to uint_8, and that this limitation could be worked around if it were important to an implementation. It has no relevance to the issue and as such comes across as creating FUD.
  https://github.com/mattreecebentley/plf_hive/issues/19
  Lewg reflector, 4/05/2022, 1:42 pm, My reply Re: [isocpp-lib-ext] Notes on P0447R20, was Re: Info+paper for hive discussion Tuesday 5th 11am pacific time (https://lists.isocpp.org/lib-ext/2022/05/23129.php)

It's kind of fun that you're allowed to set a very small maximum block size, and thus a hive can be used to simulate a traditional "rope" of fixed-capacity blocks:

```
std::hive<int> h(std::hive_limits(3, 3));
h.assign({1,2,3,4,5,6,7,8});  // [1 2 3] [4 5 6] [7 8 _]
```

It's kind of fun; but I claim that it is not *useful enough* to justify its cost, in brain cells nor in CPU time.

Imagine if `std::vector` provided this max-block-capacity API!

```
// If vector provided hive's API...
std::vector<int> v = {1,2};  // [1 2]
v.reshape({5, 8});           // [1 2 _ _ _]
v.assign({1,2,3,4,5});       // [1 2 3 4 5]
v.push_back(6);              // [1 2 3 4 5 6 _ _]
v.push_back(7);              // [1 2 3 4 5 6 7 _]
v.push_back(8);              // [1 2 3 4 5 6 7 8]
v.push_back(9);              // throws length_error, since allocating a lar
```

~~No, the real-world `vector` sensibly says that it should just keep geometrically resizing until the underlying allocator conks out.~~ In my opinion, `hive` should behave the same way. Let the *allocator* decide how many bytes is too much to allocate at once. Don't make it the container's problem.

Matt: In the real world devs often don't use vector for this exact reason. Most high-performance developers, games industry included, are Required to keep allocation amounts in check. What Arthur is suggesting has no practical value.

> Note: Discussion in SG14 (2022-06-08) suggested that maybe `hive_limits(16, 16)` would be useful for SIMD processing; but that doesn't hold up well under scrutiny. ~~The reference implementation doesn't store `T` objects contiguously~~. And even if `hive_limits(16, 16)` were useful (which it's not), that rationale wouldn't apply to `hive_limits(23, 29)` and so on.

Matt: I have evaluated my position on SIMD usage clearly in the paper in both past and present versions. The reference implementation stores objects contiguously within a block's capacity, until an erasure or splice occurs - then, any elements before and after that erasure/splice are no longer guaranteed to be contiguous. Therefore, SIMD processing is only possible when the minimum and maximum block capacity limits are a multiple of the SIMD command's width, and either (a) no erasures have occurred, or (b) an equal number of erasures and insertions have occurred, or (c) a number of consecutive erasures equal to the SIMD width, starting at a SIMD width boundary, occurred.

3.2. Max block size causes O(n) behavior

3.2.1. Example 1

> Note: The quadratic behavior shown below was *previously* present in Matt Bentley's reference implementation (as late as git commit 9486262), but was fixed in git commit 7ac1f03 by renumbering the blocks less often.

Consider this program parameterized on `N`:

```
std::hive<int> h(std::hive_limits(3, 3));
h.assign(N, 1); // [1 1 1] [1 1 1] [1 1 1] [...
while (!h.empty()) {
    h.erase(h.begin());
}
```

This loop takes $O(N^2)$ time to run! The reason is that `hive`'s active blocks are stored in a linked list, but also *numbered* in sequential order starting from 1; those "serial numbers" are required by `hive::iterator`'s `operator<`. (Aside: I don't think `operator<` should exist either, but my understanding is that that's already been litigated.)

Every time you `erase` the last element from a block (changing `[_ _ 1]` into `[_ _ _]`), you need to move the newly unused block from the active block list to the unused block list. And then you need to renumber all the subsequent blocks. Quoting P0447R20's specification of `erase`:

> *Complexity:* Constant if the active block within which `position` is located does not become empty as a result of this function call. If it does become empty, at worst linear in the number of subsequent blocks in the iterative sequence.

Since this program sets the max block capacity to 3, it will hit the linear-time case once for every three erasures. Result: quadratic running time.

3.2.2. Example 2

> Note: The quadratic behavior shown below is *currently* present in Matt Bentley's reference implementation, as of git commit ef9bad9.

Consider this program parameterized on `N`:

```
plf::hive<int> h;
h.reshape({4, 4});
for (int i=0; i < N; ++i) {
    h.insert(1);
    h.insert(2);
}
erase(h, 1); // [_ 2 _ 2] [_ 2 _ 2] [_ 2 _ 2] [...
while (!h.empty()) {
    h.erase(h.begin());
}
```

The final loop takes $O(N^2)$ time to run! The reason is that in the reference implementation, `hive`'s list of blocks-with-erasures is singly linked. Every time you `erase` the last element from a block (changing `[_ _ _ 2]` into `[_ _ _ _]`), you need to unlink the newly empty block from the list of blocks with erasures; and thanks to how we set up this snippet, the current block will always happen to be at the end of that list! So each `erase` takes $O(h.size())$ time, and the overall program takes $O(N^2)$ time.

Such "secret quadratic behavior" is caused *primarily* by how `hive` permits the programmer to set a max block size. If we get rid of the max block size, then the implementation is free to allocate larger blocks, and so we'll hit the linear-time cases geometrically less often — we'll get amortized $O(N)$ instead of $O(N^2)$.

~~Using my proposed `hive`, it will still be possible to simulate a rope of fixed-size blocks; but the programmer will have to go pretty far out of their way. There will be no footgun analogous to `std::hive<int>({1,2,3}, {3,3})`.~~

```
auto make_block = []() {
    std::hive<int> h;
    h.reserve(3);
    return h;
};
std::hive<int> h;
h.splice(make_block()); // [_ _ _]
h.splice(make_block()); // [_ _ _] [_ _ _]
h.splice(make_block()); // [_ _ _] [_ _ _] [_ _ _]
// ...
```

Matt: None of the above behaviours were present in the implementation at the time of publication. The second behaviour had already been removed from beta at the time when I reviewed Arthur's draft paper, the first no longer existed in the official release and shouldn't have been mentioned above, and is FUD. As mentioned at the start of this document, neither formed quadratic behaviour.

3.3. ~~Move semantics are arguably unintuitive~~

Matt: The transfer of capacity limits between hive instances follows much the same rules as allocator transfer - which makes sense since the two may be related in the real world. The paper has been updated with details of this. In other words, move constructors and copy constructors transfer the limits, as do swap and operator = &&, operator = & doesn't, nor does splice. With allocators in list.splice and hive.splice, the behavior is undefined if get_allocator() != x.get_allocator(). With capacity_limits in hive.splice, the behaviour is more concrete - we throw if the source blocks will not fit within the destination hive's capacity_limits.
However in the default use of hive, a user will not encounter these behaviours, because the user does not Need to specify block capacity limits, unless they desire to do so - as such, knowledge of the above transfer information is only relevant to users specifying their own custom block capacity limits.

Suppose we've constructed a hive with min and max block capacities, and then we assign into it from another sequence in various ways.

```
std::hive<int> h(std::hive_limits(3, 3));  // empty

h.assign({1,2,3,4,5});                // [1 2 3] [4 5 _]
h = {1,2,3,4,5};                      // [1 2 3] [4 5 _]

h.assign_range(std::hive{1,2,3,4,5}); // [1 2 3] [4 5 _]

std::hive<int> d = h;                 // [1 2 3] [4 5 _]
```

Matt: It should be noted here that the C++ resolution rules will change the = operation on a newly-constructed type (as ab copy-construction op instead, and since the copy constructor tranfers the block limits whereas copy assignment does not, this is the result. This is unfortunate, but the same problem occurs with allocators with propogate_on_copy_assignment == fa in that the copy assignment and copy construction will involve different transfer semantics. Removing block limits does not 'fix' the resolution scenario.

```
std::hive<int> d = std::move(h);      // [1 2 3] [4 5 _]
std::hive<int> d(h, Alloc());            // [1 2 3] [4 5 _]
std::hive<int> d(std::move(h), Alloc()); // [1 2 3] [4 5 _]

// BUT:

h = std::hive{1,2,3,4,5}; // [1 2 3 4 5]
```

Matt: In this case the compiler is selecting a move operation copy operation because that's what the C++ standard's overload say it should do with a prvalue (see notes here: https://en.cppreference.com/w/cpp/language/move_assignment). In which case it copies across the block limits along with the

which in the above case are the default block limits.
The behaviour is correct and well-defined, non-problematic. Wh
result might not be what the user expected, ultimately the abo
programming, as it involves needless construction of a tempora
and I would be surprised/disturbed to see this in production c
I would expect a reasonable programmer to use an initializer_l
Removing min/max block capacity limits does not 'fix' this sit
since allocators have the same issue - their trasnfer/non-tran
is necessarily different when copying instead of moving.

```
 // OR
std::hive<int> h2 = {1,2,3,4,5};
h = h2;                         // [1 2 3 4 5]
```

Matt: Incorrect. The above is an assignment of a named lvalue, and
rules mean it should be copied, not moved.
This is confirmed under GCC, in both debug and -O2. [1 2 3] [4 5 _]
If a compiler is doing otherwise, the compiler is operating outside
of the C++ resolution rules IMHO.

```
 // OR
std::hive<int> h2 = {1,2,3,4,5};
std::swap(h, h2);          // [1 2 3 4 5]

// BUT AGAIN:

h = std::hive<int>(std::hive_limits(3, 3));
h.splice({1,2,3,4,5});  // throws length_error
```

The `current-limits` of a hive are not part of its "value," yet they still affect its behavior in critical ways. Worse, the capacity limits are "sticky" in a way that reminds me of `std::pmr` allocators: *most* modifying operations don't affect a hive's limits (resp. a `pmr` container's allocator), but *some* operations do.

The distinction between these two overloads of `operator=` is particularly awkward:

```
h = {1,2,3,4,5};            // does NOT affect h's limits
h = std::hive{1,2,3,4,5};   // DOES affect h's limits


Matt: Again, this is the compiler correctly resolving to a move ope
```

Matt: I initially thought that this was the one portion of the document where Arthur had a point - unfortunately the point didn't only relate to the user's ability to specify a minimum/maximum block capacity specifically, but also whether the user has specified a different allocator instance in the source hive, or any other container doing the same thing. But on closer inspection it's just the compiler correctly resolving to a move operation, with the correct semantics for that move operation. Whether the user should be writing code this bad is another story - if you want to assign 1, 2, 3, 4, 5, 6 to a pre-existing hive<int>, you either write an insert command, use assign or operator = (initializer_list). Only a bad coder writes temporary hives to use with operator =, as (a) it looks confusing and (b) it wastes performance by needlessly constructing a temporary hive - if the compiler doesn't somehow manage to optimize the temporary out at -O2 (this won't happen at -O0). A non-temporary hive which is used elsewhere in code won't of course trigger the overload resolution to move.

hive limits do not (and should not) have an equivalent to `propagate_on_container_copy_assignment`, as we wish to allow for elements to be copied between hives of dissimilar limits without changing those limits, so these will never be copied on operator = (hive &) usage. I did query others regarding whether it would be worth copying hive limits on copy assignment, the status quo was the preferred solution.
Copy & move constructors for containers typically allow the user to specify a non-source allocator, however this has been considered overkill for hive limits. Status quo is that if the user wants to specify new hive limits when copy constructing, they can:
`hive<int> h(hive_limits(10,50)); h = other_hive;`
And if they want to specify new capacity limits when moving another hive, they can:
`hive<int> h(std::move(other_hive)); r.reshape(10, 50);` This avoids needless additional overloads for the copy and move constructor.

3.4. ~~`splice` is O(n), and can throw~~

Matt: As is stated in the current paper, and was stated in the draft papers for D0447R20 which were publicly available, and as is obvious from the implementation, it is at-worst O(n) in the number of memory blocks and at best O(1). In the reference implementation it will only be O(n) if the user is using user-defined block capacity limits, and if these limits differ significantly in the source.
However in the event that a hive is implemented as a vector of pointers to element blocks + metadata, splice would always be O(n) in the number of memory blocks due to the need to copy pointers and remove said pointers from the source hive. So time complexity of splice is not improved by removing the hive limit functionality.
Also as explained in above sections, splice behaviour for hive limits is largely analogous to splice behaviour for allocators. In the case of allocators, UB is caused if the allocators do not match. In the case of hive limits, throw is called but only if blocks from source do not fit within the hive_limits of destination. https://isocpp.org/files/papers/P0447R20.html#design under Additional notes on Specific functions, Splice

~~In P0447R20, `h.splice(h2)` is a "sticky" operation: it does not change `h`'s limits. This means that if `h2` contains any active blocks larger than `h.block_capacity_limits().max`, then `h.splice(h2)` will fail and throw `length_error`! This is a problem on three levels:~~

- ~~Specification speedbump: Should we say something about the state of `h2` after the throw? for example, should we guarantee that any too-large blocks not transferred out of `h2` will remain in `h2`, kind of like what happens in `std::set::merge`? Or should we leave it unspecified?~~
- ~~Correctness pitfall: `splice` "should" just twiddle a few pointers. The idea that it might actually *fail* is not likely to occur to the working programmer.~~
- ~~Performance pessimization: Again, `splice` "should" just twiddle a few pointers. But P0447R20's specification requires us to traverse `h2` looking for too-large active blocks. This adds an O(n) step that doesn't "need" to be there.~~

~~If my proposal is adopted, `hive::splice` will be "Throws: Nothing," just like `list::splice`.~~

> ~~Note: I would expect that both `hive::splice` and `list::splice` *ought* to throw `length_error` if the resulting container would exceed `max_size()` in size; but I guess that's intended to be impossible in practice.~~

3.5. `hive_limits` causes constructor overload set bloat

Every STL container's constructor overload set is "twice as big as necessary" because of the duplication between `container(Args...)` and `container(Args..., Alloc)`. `hive`'s constructor overload set is "four times as big as necessary" because of the duplication between `container(Args...)` and `container(Args..., hive_limits)`.

Matt: All additional overloads required by hive_limits are implemented by way of constructor delegation, meaning there is no code bloat. This is not only specified in the implementation, but in the Technical Specification of Hive itself.

P0447R20 `hive` has 18 constructor overloads. My proposal removes 7 of them. (Of course, we could always eliminate these same 7 constructor overloads without doing anything else to P0447R20. If this were the *only* complaint, my proposal would be undermotivated.)

Analogously: Today there is no constructor overload for `vector` that sets the capacity in one step; it's a multi-step process. Even for P0447R20 `hive`, there's no constructor overload that sets the overall capacity in one step — even though overall capacity is more important to the average programmer than min and max block capacities!

```
// Today's multi-step process for vector
std::vector<int> v;
v.reserve(12);                  // [_ _ _ _ _ _ _ _ _ _ _ _]
v.assign({1,2,3,4,5,6});  // [1 2 3 4 5 6 _ _ _ _ _ _]

// Today's multi-step process for hive
std::hive<int> h;
h.reshape(std::hive_limits(12, h.block_capacity_hard_limits().max));
h.reserve(12);                  // [_ _ _ _ _ _ _ _ _ _ _ _]
h.reshape(h.block_capacity_hard_limits());
h.assign({1,2,3,4,5,6});  // [1 2 3 4 5 6 _ _ _ _ _ _]

// Today's (insufficient) single-step process for hive
// fails to provide a setting for overall capacity
std::hive<int> h({1,2,3,4,5,6}, {3, 5});
                                // [1 2 3 4 5] [6 _ _]
```

If my proposal is adopted, the analogous multi-step process will be:

```
std::hive<int> h;
h.reshape(12, 12);          // [_ _ _ _ _ _ _ _ _ _ _ _]
h.assign({1,2,3,4,5,6}); // [1 2 3 4 5 6 _ _ _ _ _ _]
```

3.6. `hive_limits` introduces unnecessary UB

Matt: The issues Arthur has raised here were previously discussed in a LEWG meeting and resolved via the block_capacity_hard_limits() static member function, which is detailed in the paper and was in the publicly available draft version of that paper for a long time. https://isocpp.org/files/papers/P0447R20.html#design under Additional notes on Specific functions, block_capacity_hard_limits()

[D0447R20] currently says ([hive.overview]/5):

> If user-specified limits are supplied to a function which are not within an implementation's *hard limits*, or if the user-specified minimum is larger than the user-specified maximum capacity, the behaviour is undefined.

In Matt Bentley's reference implementation, this program will manifest its UB by throwing `std::length_error`.

The following program's behavior is always undefined:

```
std::hive<int> h;
h.reshape({0, SIZE_MAX}); // UB! Throws.
```

Worse, the following program's definedness hinges on whether the user-provided max `1000` is greater than the implementation-defined `hive<char>::block_capacity_hard_limits().max`. The reference implementation's `hive<char>::block_capacity_hard_limits().max` is `255`, so this program has undefined behavior (Godbolt):

```
std::hive<char> h;
h.reshape({10, 1000}); // UB! Throws.
```

~~There are two problems here. The first is trivial to solve: P0447R20 adds to the set of unnecessary library UB. We could fix that by simply saying that the implementation must *clamp the provided limits* to the hard limits; this will make `h.reshape({0, SIZE_MAX})` well-defined, and make `h.reshape({10, 1000})` UB only in the unlikely case that the implementation doesn't support *any* block sizes in the range [10, 1000]. We could even mandate that the implementation *must* throw `length_error`, instead of having UB.~~

~~The second problem is bigger: `hive_limits` vastly increases the "specification surface area" of `hive`'s API.~~

- ~~Every constructor needs to specify (or UB) what happens when the supplied `hive_limits` are invalid.~~
- ~~`reshape` needs to specify (or UB) what happens when the supplied `hive_limits` are invalid.~~
- ~~`splice` needs to specify what happens when the two hives' *`current_limits`* are incompatible.~~
- ~~The special members — `hive(const hive&)`, `hive(hive&&)`, `operator=(const hive&)`, `operator=(hive&&)`, `swap` — need to specify their effect on each operand's *`current_limits`*. For example, is `operator=(const hive&)` permitted to preserve its left-hand side's *`current_limits`*, in the same way that `vector::operator=(const vector&)` is permitted to preserve the left-hand side's capacity? The Standard needs to specify this.~~
- ~~`reshape` needs to think about how to restore the hive's invariants if an exception is thrown. (See below.)~~

~~All this extra specification effort is costly, for LWG and for vendors. My "Proposed Wording" is mostly deletions.~~

MATT: If you look at the code - which Arthur has in detail, since he has submitted dozens of Issues on the hive github page - the additional implementation effort is minimal, at best. Additional constructors via delegation, the 2 single-line functions for obtaining hive limits, the reshape function. And since most major vendors have suggested that they will either use the existing hive codebase, or use it as a reference or starting point, there is no real additional cost to them. The work is done. This latter info was publicly available on the LEWG reflector.
See: Re: [isocpp-lib-ext] Is hive priority worth the cost? (Matt Bentley, Mon, 11 Apr 2022 12:51) (https://lists.isocpp.org/lib-ext/2022/04/22987.php)

When I say "`reshape` needs to think about how to restore the hive's invariants," I'm talking about this example:

```
std::hive<W> h({1,2,3,4,5}, {4,4}); // [1 2 3 4] [5 _ _ _]
h.reshape({5,5});  // suppose W(W&&) can throw
```

Suppose `W`'s move constructor is throwing (and for the sake of simplicity, assume `W` is move-only, although the same problem exists for copyable types too). The hive needs to get from `[1 2 3 4] [5 _ _ _]` to `[1 2 3 4 5]`. We can start by allocating a block `[_ _ _ _ _]` and then moving the first element over: `[1 2 3 4] [5 _ _ _ _]`. Then we move over the next element, intending to get to `[1 2 3 _] [5 4 _ _ _]`; but that move construction might throw. If it does, then we have no good options. If we do as `vector` does, and simply deallocate the new buffer, then we'll lose data (namely element `5`). If we keep the new buffer, then we must update the hive's *`current_limits`* because a hive with limits `{4,4}` cannot hold a block of capacity 5. But a hive with the new user-provided limits `{5,5}` cannot hold a block of capacity 4! So we must either lose data, or replace `h`'s *`current_limits`* with something "consistent but novel" such as `{4,5}` or `h.block_capacity_hard_limits()`. In short: May a failed operation leave the hive with an "out-of-thin-air" *`current_limits`*? Implementors must grapple with this kind of question in many places.

§ 3.7. Un-criticism: Batches of input/output

The `reshape` and `hive_limits` features are not mentioned in any user-submitted issues on [PlfColony], but there is one relevant user-submitted issue on [PlfStack]:

> If you know that your data is coming in groups of let's say 100, and then another 100, and then 100 again etc but you don't know when it is gonna stop. Having the size of the group set at 100 would allow to allocate the right amount of memory each time.

In other words, this programmer wants to do something like this:

```
// OldSmart
int fake_input[100] = {};
std::hive<int> h;
h.reshape(100, 100);  // blocks of size exactly 100
while (true) {
    if (rand() % 2) {
        h.insert(fake_input, fake_input+100);  // read a whole block
    } else {
```

```
        h.erase(h.begin(), std::next(h.begin(), 100));  // erase and "unuse
    }
}
```

If the programmer doesn't call `reshape`, we'll end up with `h.capacity() == 2*h.size()` in the worst case, and a single big block being used roughly like a circular buffer. This is actually not bad.

A programmer who desperately wants the exactly-100 behavior can still get it, after this patch, by working only a tiny bit harder:

```
// NewSmart
int fake_input[100] = {};
std::hive<int> h;
while (true) {
    if (rand() % 2) {
        if (h.capacity() == h.size()) {
            std::hive<int> temp;
            temp.reserve(100);
            h.splice(temp);
        }
        h.insert(fake_input, fake_input+100);  // read a whole block
    } else {
        h.erase(h.begin(), std::next(h.begin(), 100));  // erase and "unuse
    }
}
```

Matt: That is... not the least bit tiny. That's the sort of code I would expect to have to make for something which is an edge-case functionality in a std::library container, not common usage stuff. The amount of code necessary to support max block capacity limits is not that much larger either.

I have benchmarked these options ([Bench]) and see the following results. (`Matt` is Matthew Bentley's original `plf::hive`, `Old` is my implementation of P0447, `New` is my implementation of P2596; `Naive` simply omits the call to `reshape`, `Smart` ensures all blocks are size-`N` using the

corresponding approach above.) With N=100, no significant difference between the `Smart` and `Naive` versions:

```
$ bin/ubench --benchmark_display_aggregates_only=true --benchmark_repetitic
2022-06-10T00:32:11-04:00
Running bin/ubench
Run on (8 X 2200 MHz CPU s)
CPU Caches:
  L1 Data 32 KiB (x4)
  L1 Instruction 32 KiB (x4)
  L2 Unified 256 KiB (x4)
  L3 Unified 6144 KiB (x1)
Load Average: 2.28, 2.14, 2.06
BM_PlfStackIssue1_MattSmart_mean        93298 ns        93173 ns
BM_PlfStackIssue1_MattNaive_mean        93623 ns        93511 ns
BM_PlfStackIssue1_OldSmart_mean        114478 ns       114282 ns
BM_PlfStackIssue1_OldNaive_mean        113285 ns       113124 ns
BM_PlfStackIssue1_NewSmart_mean        128535 ns       128330 ns
BM_PlfStackIssue1_NewNaive_mean        116530 ns       116380 ns
```

With N=1000 (`Matt` can't handle this because `h.block_capacity_hard_limits().max < 1000`), again no significant difference:

```
$ bin/ubench --benchmark_display_aggregates_only=true --benchmark_repetitic
2022-06-10T00:33:12-04:00
Running bin/ubench
Run on (8 X 2200 MHz CPU s)
CPU Caches:
  L1 Data 32 KiB (x4)
  L1 Instruction 32 KiB (x4)
  L2 Unified 256 KiB (x4)
  L3 Unified 6144 KiB (x1)
Load Average: 2.13, 2.17, 2.08
BM_PlfStackIssue1_OldSmart_mean       973149 ns       972004 ns
BM_PlfStackIssue1_OldNaive_mean       970737 ns       969565 ns
BM_PlfStackIssue1_NewSmart_mean      1032303 ns      1031035 ns
BM_PlfStackIssue1_NewNaive_mean      1011931 ns      1010680 ns
```

Matt: These micro-benchmarks do not take into account the usage patterns of the container. In the benchmarks which I did regarding maximum block capacity limits, that were explained in

detail on the LEWG reflector, there were no performance advantages to maximum block capacities greater than 16384 elements, only greater memory waste. There is a lot wrong with the above. But all the information required to understand performance related to block capacities is already publicly available on the reflector, and this data is what informed the removal of the hive priority template parameter in the first place.
See: Re: [isocpp-lib-ext] Is hive priority worth the cost? (Matt Bentley, Sun, Apr 10, 2022 at 8:51 PM) (https://lists.isocpp.org/lib-ext/2022/04/22987.php)

Matt: SUMMARY

Points in favour of retaining status quo:

1. The library/implementation doesn't know the cache-line size, and users may want their element blocks to be multiples of cache-line capacity in order to improve read locality.
2. The library/implementation doesn't know the user's erasure/insertion patterns for their data, and the efficiency of hive traversal and memory waste associated with that is affected by block capacities. eg. Users inserting/erasing in fixed amounts would likely want blocks that match that amount (or multiples of that amount).
3. Bigger max block capacity != better, due to erasures. Blocks are removed from the iterative sequence once empty of non-erased elements, and with a high erasure percentage you will end up with more gaps between elements, and more wasted memory, statistically-speaking, the larger your block capacity is. Gaps between elements affect cache locality during iteration and performance. And the memory waste can hit embedded and high-performance devs hard.
4. Bigger max block capacity != better, due to cache limits. Once your block is >= the size of your cache line, you don't get any greater locality, only fewer allocations/deallocations upon insertion/erasure. This is confirmed in the benchmarks mentioned earlier.
5. Maximum block capacities are necessary to prevent over-allocation. This is already a problem we have with vector. Particularly for high-memory-demand environments like game engines and the like, not being able to control the size of the next allocation is not an option.
6. Some allocators may only allocate in specific chunks, and this feature aids that (although the user would need to look at the container code to figure out the total amount of allocation, unless we add in the function (from colony) to do it for them).
7. We cannot necessarily anticipate all user needs and should aim for more flexibility of usage within reason, not less.

8. Some users may wish to use hive for SIMD processing, the usage of which is explained earlier.
9. Blocks limits are always going to be in an implementation regardless of whether a user specifies them. On the minimum side it makes sense to have a reasonable minimum which isn't lower in size than the amount of memory used for that first block's metadata. On the maximum side this is determined (in this implementation) by the bit-depth of the jump-counting skipfield. A 16-bit skipfield means a maximum jump of 65535 elements within the block, so the *hard* max block limit becomes 65536 (note: if a skip goes further than the block, a separate skip is recorded at the start of the next block). Higher bit-depths waste memory and do not perform better, for many reasons (including points 3 & 4 above). So the extra code to support user block limits is nearly incidental.
10. This is the one feature which actual users have repeatedly thanked me for because they find it useful. It is quite bizarre to me that someone would choose to remove it on grounds of technical personal aesthetics.

Points in favour of removing this feature (which have not been proven false):

1. None of the other containers have it - well, deque and vector have capacity, but list doesn't. map and set have keys, but vector, deque and list don't. List has splice and none of the others have it. Hive has capacity limits, and none of the others have it.
2. The presense of limits increases the number of constructors - but, all of those additional overloads are achievable via delegation, so there is no code bloat.
3. Increases implementation load - as mentioned the increase is barely anything. All the Actual work goes into the specifics of the container and the three core design aspects. Since two major vendors have stated they'll likely fork the reference implementation, and the other is likely to use it as a reference, the minimal additional load is *already done*.

This paper has been a waste of committee time, my time, and has served no-one. I would like you to dismiss it - Matt